Robot Project Final Report

Carmen Lin (cl2624), Gina Liu (gl528), Anish Dhawan (akd82)

Robot Design and Strategy Overview

Our robot, pictured left, was programmed with a simple strategy. It followed a set path to the dispersed blocks area, picked up as many blocks as it could, and used sensors to avoid other robots and the field boundary. We focused on design consistency rather than complexity, so the robot could repeat the same actions each time with only minimal real-time decision making.

The robot was driven by two DC motors, each connected to its own H-bridge. We installed a rubber band on the wheel to help prevent it from slipping while moving. The robot had a frame made of cardboard and acrylic that protruded past the chassis, which also served as a cube collector. This design gave the robot a low center of gravity, which made it more stable and helped during deadlocks.

An Arduino UNO controlled the robot and was connected to the motors and sensors (Appendix B). Two QTI line sensors at the front detected the black border, keeping the robot within the field. An ultrasonic sensor at the front measured the distance to other robots. We used PWM signals from the Arduino to control the motor speed and direction through the H-bridge. Two battery packs powered everything, and all components shared a common ground.

The software was primarily hardcoded. At the start of the match, the robot executed a sequence of timed movements and turns that guided it to the middle (block collection area). Once the path was completed, the robot entered a continuous control loop. Within the loop, the robot exhibited these behaviors: boundary detection powered by the QTIs, obstacle avoidance aided by the ultrasonic sensor, and normal forward motion. A timer automatically stopped the robot after 61 seconds in compliance with the competition rules. More details can be found in Appendix D and E.

Overall, we focused on making our robot simple, consistent, and reliable. With a strong frame, a basic electrical setup, and a hard-coded control system that used sensors for obstacles and borders, the robot could repeat its task to collect as many blocks as possible during the competition.

Design Process Reflection

For the first milestone, we proposed to use fans under the robot to sweep blocks toward and under the chassis, preventing them from escaping easily. We also planned to add arms to help gather blocks and use sensors to detect blocks and other robots.

The second milestone tested the robot's ability to move forward, backward, left, and right through a course. We measured how long it took to move the required distances and tried different timings to turn 90° left and right. Despite the relatively simple process, we noticed that one motor spun faster than the other, causing the robot to veer off course. However, after attaching both motors to the same power source, the issue was resolved as they now received equal voltages.

For milestone 3, which involved color sensing and border detection, we encountered significant issues. The color sensor could distinguish black and blue when the robot was stationary, but when the robot was moving, the sensor occasionally detected a shorter black border than expected, causing it to confuse black for blue. After trial and error and many changes to the timing and the code, we finished the milestone. Later, we learned we could use QTIs to detect the black border, which we utilized in our final design.

The last milestone, cube clearing, went smoothly. We used cardboard arms and reused code from our first milestone to hardcode the robot's path for collecting blocks. But because the cardboard wasn't precise, the robot became unbalanced and swerved off its straight path. We fixed this by trimming one side of the arms to even out the weight.

Since we focused on completing the milestones, the robot's mechanical design was overshadowed, leading to numerous issues. At first, we wanted to optimize speed, but we accidentally bought smaller wheels, which made it slower and prevented us from storing blocks under the chassis, so we returned to our original wheels. After dropping the fan idea, we tested an arm system with a front gate that would close when the robot moved backward to keep blocks inside. Linear actuators were too expensive, so we tested a rack-and-pinion system with a motor, but the motor didn't provide enough torque to move the rack. We ended up using simple arms and sensors to collect blocks and detect colors, but measurement errors meant the arms didn't fit right. Instead, we used hot glue to attach the acrylic arm pieces, assembling them differently from the planned assembly (Appendix C). Even though the process was not linear, the final robot performed better than expected in the competition.

Competition Analysis

On competition day, our robot's performance was mixed. We lost our first two matches in the round-robin stage, but after we changed the robot's starting angle, it performed significantly better and won the next four rounds. However, we lost the seventh round and didn't qualify for the single-elimination bracket.

Despite the disappointment, our main goal was to simply build a robot that could reliably collect blocks. With that in mind, we accomplished our goals and performed better than expected, winning against numerous teams during the competition.

One of our robot's biggest strengths was how sturdy it was. In almost every match, we ended up in a deadlock with another robot. Instead of being pushed back, our robot could push the other robot out of the way, sometimes even off the playing field, then recalibrate and keep collecting blocks. It was also reliable and experienced no major mechanical failures during matches.

Another advantage was the robot's consistent block collection, due to its hard-coded programming. But this also proved to be a weakness, since the robot needed tuning before it could work effectively, as mentioned previously. Another downside was its speed. In our first loss, the opponent collected blocks faster, so there were none left by the time we reached the middle.

Overall, we were pleased with how the robot performed. It bounced back from an early setback, won four matches in a row, and almost made it to the single-elimination bracket. Most importantly, it met our goals and showed impressive strength and reliability during the tournament.

Conclusions

Looking back on this project, one of the biggest lessons was learning how to adapt when things did not go as planned. Many of our original ideas either did not work as expected or had to be completely changed, so we were constantly adjusting and improving the robot throughout the semester. Even when setbacks became frustrating, we learned how to keep improving our ideas and work through problems together.

If we could do the project again with the same budget and constraints, we would start designing much earlier. We spent too much time on each milestone separately and lost track of the overall robot design until late in the semester. This meant we had to assemble and redesign big parts of the robot in the last week, which was stressful and left little time for testing. Better delegation would have helped our workflow. Even though we often met in the lab, not everyone always had a clear task, so organizing responsibilities more effectively could have made us more efficient.

For students taking this class next year, our main advice is to learn the coding concepts as soon as they are taught, instead of waiting until the milestones to figure them out. Many of our delays occurred because we were learning, debugging, and completing the milestone simultaneously. It's also important to leave enough time for testing, since many problems only show up when the robot is fully built and running. Finally, keeping your design simple and reliable usually works better than trying to build something too complicated that might be hard to finish on time and within budget.

Appendix A

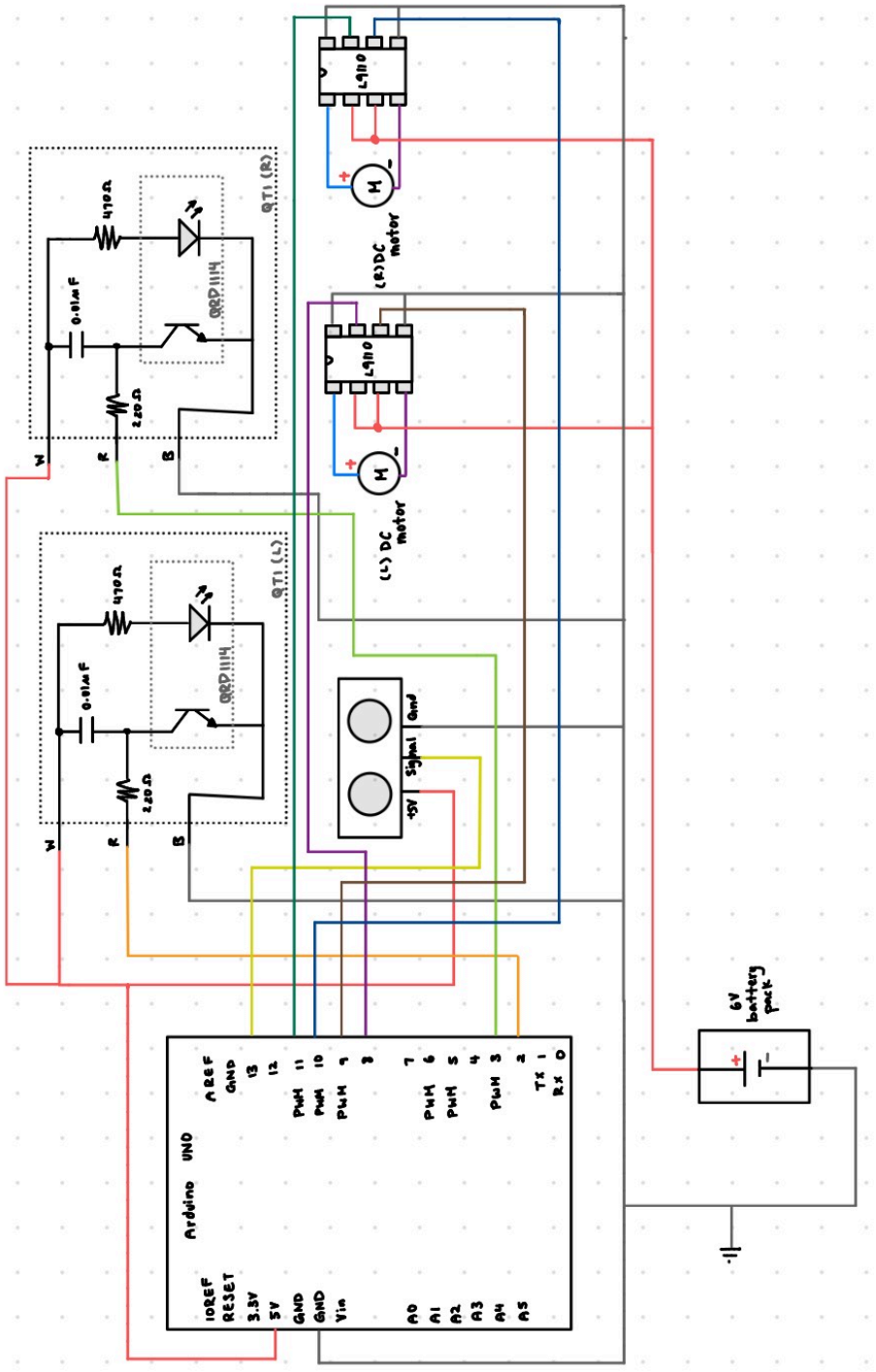
Bill of Materials

Part	Source	Quantity	Price Per Part	Total
1.63" Rubber Wheels	McMaster	2	$2.32	$4.64
20 Degree Pressure Angle Plastic Gear	McMaster	1	$5.71	$5.71
Acrylic Arms	Laser Cut	1	see RPL Guide	$15.42
0.354" Cylindrical Roller	3D Print (PLA)	1	see RPL Guide	$1.43
20 Degree Pressure Angle Gear Rack	3D Print (PLA)	1	see RPL Guide	$4.29
Rack Holder	3D Print (PLA)	1	see RPL Guide	$3.66

Total	$35.15

Appendix B

Circuit Diagram



Appendix C

Cad Files and Drawings

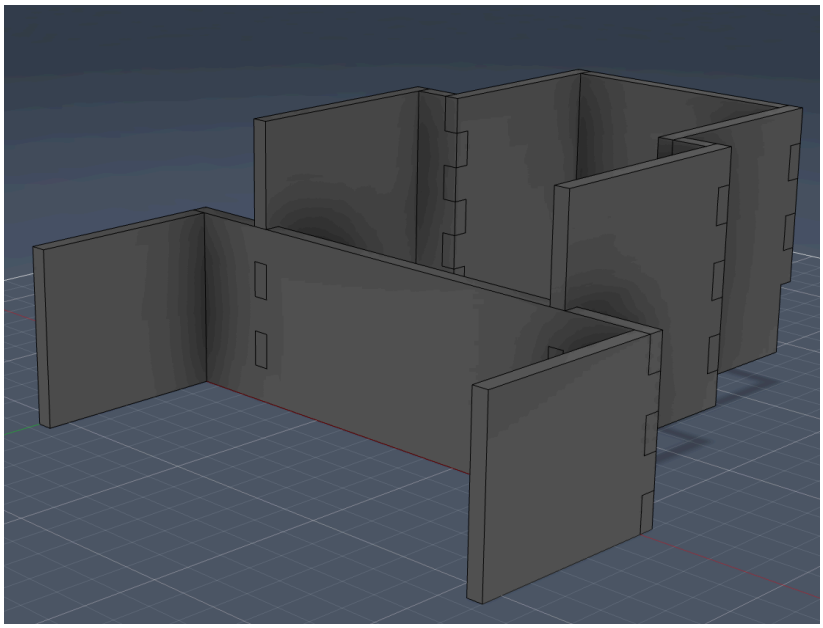


Figure 1: CAD assembly of original arm configuration.

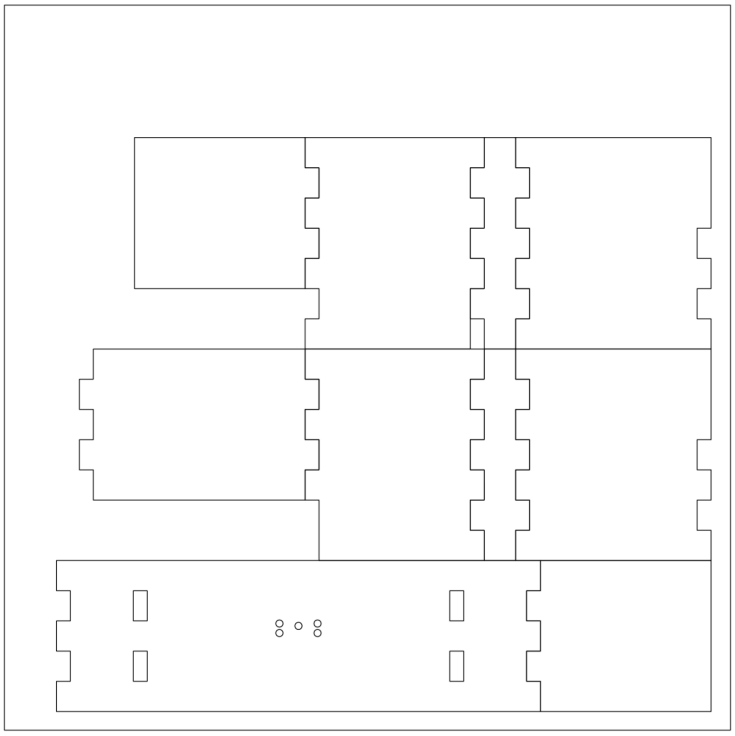


Figure 2: DXF file of CAD arms sent for laser cutting.

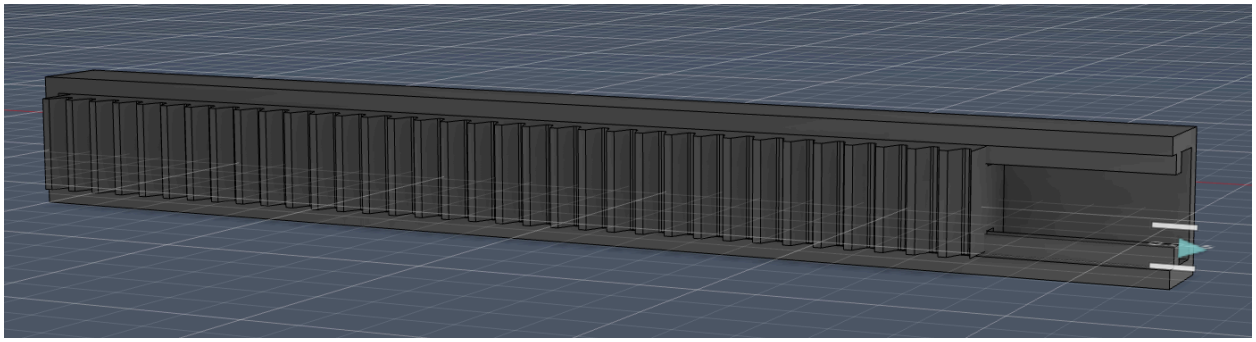


Figure 3: CAD assembly of rack and rack holder configuration.

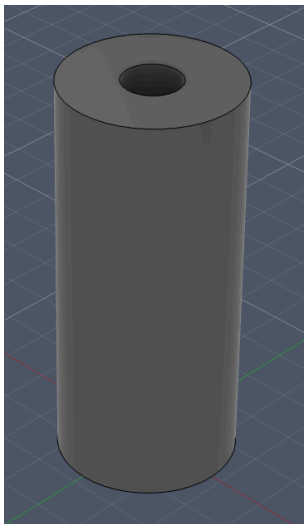


Figure 4: CAD of cylindrical roller part.

Volume and Weight Calculations

All volumes obtained are from Fusion 360. A density of 1.25 g/cm^2 is used for PLA. 75% of the weight is used to calculate the cost as recommended on the RPL guide.

Rack:

Volume	8.780 cm^3

$$8.78 \cdot 1.25 \cdot 0.75 = 8.23125$$ (Weight used in grams)

Rack Holder:

Volume	7.095 cm^3

$$7.095 \cdot 1.25 \cdot 0.75$$

$$= 6.6515625$$ (Weight used in grams)

Cylindrical Roller:

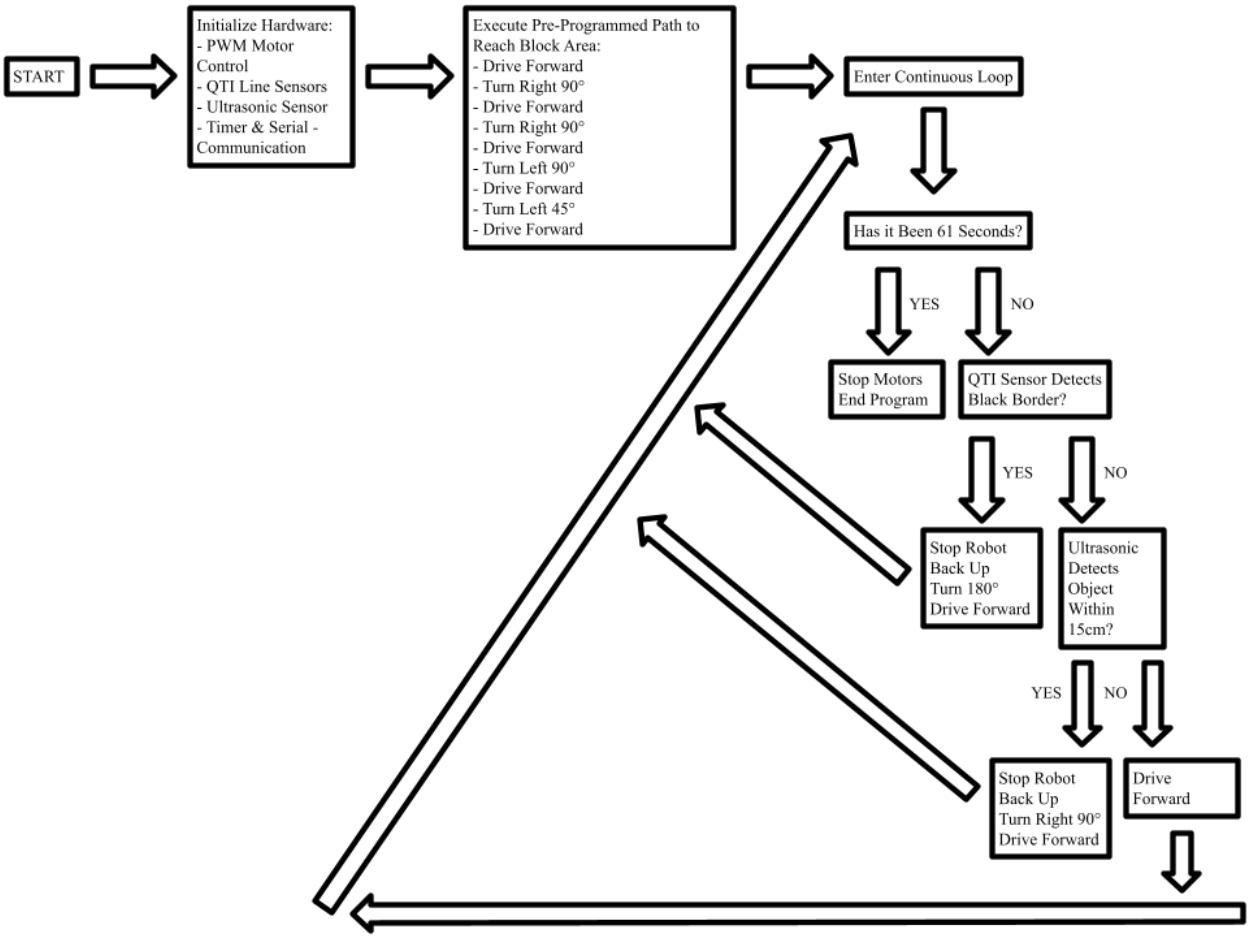
Volume	1.149 cm^3

$$1.149 \cdot 1.25 \cdot 0.75$$

$$= 1.0771875$$ (Weight used in grams)

Appendix D

Flowchart



Appendix E

Code

```
/*
Group Number 22
Members: Carmen Lin (cl2624), Gina Liu (gl528), Anish Dhawan (akd82)
*/
// ---------------- VARIABLES ----------------
// Approximate time needed to turn 1 degree (ms)
float tperdegree = 5.5;
// Approximate time needed to drive 1 foot straight (ms)
float straighttperft = 1700;
// Stores ultrasonic pulse period
volatile int period;
// Stores Timer1 value
volatile unsigned int timerVal;
// ---------------- PWM SETUP ----------------
// Initializes PWM for motor speed control
void initPWM(){
  // Set PB1 and PB2 as outputs
  DDRB |= 0b00000110;

  // Clear Timer1 registers
  TCCR1A = 0;
  TCCR1B = 0;
  // Configure PWM mode
  TCCR1A = 0b10100001;
  TCCR1B = 0b00001011;
  // Initial motor speeds
  OCR1A = 200;
  OCR1B = 200;
}
// ---------------- ROBOT MOVEMENT ----------------
// Drives robot forward
void drive_forward(){
  // Set motor direction bits for forward motion
  PORTB = (PORTB & 0b11110000) | 0b00000101;
  // Set equal motor speeds
```

```
  OCR1A = 200;
  OCR1B = 200;
}

// Turns robot left
void drive_left(int speed) {
  // Set motor directions for left turn
  PORTB = (PORTB & 0b11110000) | 0b00000110;
  // Reduce left motor speed
  OCR1A = speed;
  // Keep right motor at full speed
  OCR1B = 200;
}
// Turns robot right
void drive_right(int speed) {
  // Set motor directions for right turn
  PORTB = (PORTB & 0b11110000) | 0b00001001;
  // Keep left motor at full speed
  OCR1A = 200;
  // Reduce right motor speed
  OCR1B = speed;
}

// Stops robot movement
void stop() {
  // Turn off all motor direction bits
  PORTB = (PORTB & 0b11110000) | 0b00000000;
}
// Turns the robot right by specified degrees
void Turn_Right_Deg(int d){
  // Set the right motor to a speed of 175
  drive_right(175);
  // Delay loop to control turning amount
  for (int i = 0; i < d * 5.5; i++) {
    _delay_ms(1);
  }
}
// Turns the robot left by specified degrees
void Turn_Left_Deg (int d){
```

```
  // Set the left motor to a speed of 175
  drive_left(175);
  // Delay loop to control turning amount
  for (int i = 0; i < d * 6.5; i++) {
    _delay_ms(1);
  }
}
// Drives robot backward
void drive_backward() {
  // Turn robot 180 degrees
  Turn_Right_Deg(180);
  // Drive forward after turning around
  drive_forward();
  // Drive forward duration
  _delay_ms(2000);
}
// ---------------- ULTRASONIC SENSOR INTERRUPT ----------------
// Interrupt for ultrasonic pulse timing
ISR(PCINT0_vect){
  // If pulse starts
  if (PINB & 0b00100000) {
  // Reset timer
  TCNT1 = 0;
  }
  // If pulse ends
  else {
  // Store pulse duration
  timerVal = TCNT1;
}
}
// ---------------- TIMER VARIABLES ----------------
// Tracks elapsed seconds
volatile int seconds = 0;
// Flag to stop robot after 61 seconds
volatile int stop_flag = 0;
// ---------------- TIMER1 SETUP ----------------
// Initializes Timer1 interrupt
void initTimer1(){
  // Clear timer registers
```

```
  TCCR1A = 0;
  TCCR1B = 0;
  // Reset timer count
  TCNT1 = 0;
  // Compare value for 1 second
  OCR1A = 15624;
  // Enable CTC mode
  TCCR1B |= (1 << WGM12);
  // Enable compare interrupt
  TIMSK1 |= (1 << OCIE1A);
  // Set prescaler to 1024
  TCCR1B |= (1 << CS12) | (1 << CS10);
}
// Runs every second
ISR(TIMER1_COMPA_vect){
  // Increase second counter
  seconds++;
  // Stop robot after 61 seconds
  if (seconds >= 61) {
  stop_flag = 1;
  // Stop Timer1
  TCCR1B = 0;
  }
}
// ---------------- QTI SENSOR SETUP ----------------
// Initialize left QTI sensor
void initLQTI() {
  // Set PD2 as input
  DDRD &= ~(1 << PD2);
  // Enable pull-up resistor
  PORTD |= (1 << PD2);
}
// Initialize right QTI sensor
void initRQTI() {
  // Set PD3 as input
  DDRD &= ~(1 << PD3);
  // Enable pull-up resistor
  PORTD |= (1 << PD3);
}
```

```
// Reads left QTI sensor
int readleftQTI() {
  // Return 1 if black border detected
  return (PIND & (1 << PD2)) ? 1 : 0;
}
// Reads right QTI sensor
int readrightQTI() {
  // Return 1 if black border detected
  return (PIND & (1 << PD3)) ? 1 : 0;
}
// ---------------- ULTRASONIC SENSOR ----------------
// Reads ultrasonic distance in cm
  int  readUltrasonicCM() {
  // Set PB5 as output
  DDRB |= 0b00100000;
  // Clear trigger
  PORTB &= ~0b00100000;
  _delay_ms(2);
  // Send trigger pulse
  PORTB |= 0b00100000;
  _delay_ms(1);
  // End trigger pulse
  PORTB &= ~0b00100000;
  // Set PB5 as input
  DDRB &= ~0b00100000;
  int wait = 30000;
  // Wait for echo pulse
  while (!(PINB & 0b00100000) && wait > 0) {
    wait--;
    }
  // Timeout protection
  if (wait == 0) {
    return 999;
    }
  int count = 0;
  // Measure echo pulse duration
  while ((PINB & 0b00100000) && count < 30000) {
    count++;
    }
```

```
  // Timeout protection
  if (count >= 30000) {
    return 999;
    }
  // Convert pulse count to cm
  return count / 58;
}
// ---------------- MAIN PROGRAM ----------------
int main(void){
      // Set PB0-PB3 as outputs
      DDRB = DDRB | 0b00001111;
      // Set PD2 as input
      DDRD = DDRD & 0b11111011;
      // Start serial monitor
      Serial.begin(9600);
      // Initialize sensors
      initRQTI();
      initLQTI();
// -------- INITIAL HARDCODED PATH --------
      drive_forward();
      _delay_ms(3000);
      Turn_Right_Deg(90);
      drive_forward();
      _delay_ms(2700);
      Turn_Right_Deg(90);
      drive_forward();
      _delay_ms(5600);
      Turn_Left_Deg(90);
      drive_forward();
      _delay_ms(1500);
      Turn_Left_Deg(45);
      drive_forward();
      _delay_ms(2700);
// -------- MAIN CONTROL LOOP --------
      while(1) {
        // Stop robot after 61 seconds
        if (stop_flag == 1) {
          PORTB = 0b00000000;
        }
```

```
        else {
          // Border detection behavior
          if (readrightQTI() == 1 || readleftQTI() == 1 ){
            stop();
            drive_backward();
            _delay_ms(300);
            Turn_Right_Deg(180);
            drive_forward();
            _delay_ms(1000);
          }
          // Obstacle avoidance behavior
          else if(readUltrasonicCM() < 15){
            stop();
            drive_backward();
            _delay_ms(300);
            Turn_Right_Deg(90);
            drive_forward();
            _delay_ms(200);
          }
          // Normal forward motion
          else{
            drive_forward();
      }
    }
  }
}
```